SIMATS ENGINEERING

Department of Information Technology

CSA37 – Software Testing

ASSIGNMENT 2 – CO2

Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator

CO2: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Student Name	S Deepak	**Register Number**	1921260077
Department / Section	B.Tech IT	**Academic Year / Semester**	2026-2030
Faculty Name	R Kesavan	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	02-11-2026	**Date of Evaluation**	

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
1	Select Test Case Design Strategies	5		
2	Evaluate Random Testing	2		
3	Design and Evaluate Equivalence Classes	6		
4	Design and Evaluate Boundary Value Tests	4		
5	Apply Other Black-Box Techniques	3		
6	Design and Evaluate State-Transition Tests	3		
7	Construct the Control Flow Graph	4		
8	Evaluate Path Testing and Cyclomatic Complexity	4		
9	Judge Test Adequacy and Coverage	3		
10	Compare Black-Box and White-Box Effectiveness	4		
11	Recommend and Defend the Final Test Suite	4		

Task	Task Title	Max Marks	Marks Awarded	Remarks
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of the Original Suite	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	**50**		**Percentage:**

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task.

Task	Evidence expected	Status	Note
1	A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.		
2	A reasoned evaluation of the usefulness and limits of random testing for shipCharge.		
3	A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.		
4	A justified boundary value test design with expected results and a defended choice of BVA form.		
5	A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.		
6	A state-transition model with justified valid and invalid transition tests.		
7	A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.		
8	A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.		
9	A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.		
10	A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.		
11	A technically justified final test suite supported by black-box, white-box and coverage evidence.		
12A–D	Impact analysis, evaluation of the original suite, adaptation or redesign, and evidence-based defence		

TEST DESIGN EVIDENCE CHAIN (Tasks 3–11)

Evidence item	Status	Observation
Equivalence classes derived from the specification		
Boundary values and expected results calculated correctly		
Decision table and state-transition model with derived tests		
Control flow graph, V(G) and basis paths shown with derivation		
Coverage measured on the student's own tests		

Evidence item	Status	Observation
Defects found traced to the technique that found them		
Final suite justified against alternatives		
Figures and conclusions consistent		

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All 12 tasks (including 12A–12D) answered	☐ Yes ☐ Partly ☐ No
Expected results calculated from the specification, not from the code	☐ Yes ☐ Partly ☐ No
Equivalence classes, boundaries and decision table shown with tests	☐ Yes ☐ Partly ☐ No
Control flow graph and cyclomatic complexity derived	☐ Yes ☐ Partly ☐ No
Coverage and adequacy criterion justified	☐ Yes ☐ Partly ☐ No
State-transition model and tests included	☐ Yes ☐ Partly ☐ No
Tables, figures and diagrams included and readable	☐ Yes ☐ Partly ☐ No
Cross-references and table/figure numbering correct	☐ Yes ☐ Partly ☐ No
Work consistent throughout	☐ Yes ☐ Partly ☐ No
No unfilled template placeholders	☐ Yes ☐ Partly ☐ No

Software Testing (CSA37)
ASSIGNMENT 2

CO2 Coverage: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator (Total Marks - 50)

Duration:1week

Scenario — Courier Shipping-Charge Calculator (SwiftShip)

A courier company is launching SwiftShip, an online service that calculates the shipping charge for a parcel and tracks it until delivery. More than 40,000 parcels are booked every day. An undercharge causes revenue loss; an overcharge causes customer complaints and refunds.

You are the Test Designer. You must design test cases, evaluate their effectiveness and defend the final test suite that will be used before every release.

Shipping-charge rules (specification extract):

Rule	Specification
S1	Weight (kg, up to one decimal place) must be greater than 0 and not more than 50. Otherwise the booking is rejected as INVALID.
S2	Charge is calculated for every started kilogram: Zone A ₹40/kg, Zone B ₹60/kg, Zone C ₹90/kg. Any other zone is INVALID.
S3	Express delivery adds 50 % to the charge. Express is allowed only up to 30 kg; above 30 kg the request is NOT_ALLOWED.
S4	Customers who booked 10 or more parcels in the current month receive a 10 % discount on the final charge.

Shipping-charge module – Pseudocode under test:

```
function shipCharge(weight, zone, express, parcels)
1   if weight <= 0 or weight > 50 then
2       return INVALID
3   rate = 40
4   if zone == "B" then
5       rate = 60
6   else if zone == "C" then
7       rate = 90
8   charge = 0
9   w = weight
10  while w > 0 do
11      charge = charge + rate
```

```
12      w = w - 1
13  if express then
14      if weight > 30 then
15          return NOT_ALLOWED
16      charge = charge * 1.5
17  if parcels >= 10 then
18      charge = charge * 0.9
19  return charge
```

Parcel tracking states:

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED.

A parcel can move from OUT_FOR_DELIVERY to FAILED_ATTEMPT, and from FAILED_ATTEMPT back to OUT_FOR_DELIVERY (maximum two retries) or to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Design Challenge:

SwiftShip will add a fuel surcharge and weekend delivery, and the shipping-charge code will be modified.

Re-evaluate your test suite and determine whether it should be retained, modified or redesigned.

TASK 1 – Select Test Case Design Strategies 5 Marks
CO2 | Bloom's Level: K5 – Evaluate

SwiftShip has a calculation module (shipCharge) and a tracking module.

Which test design strategy will find the most important defects in each module with the least effort?

Your answer must include:

Discuss:

- **The difference between black-box and white-box test design strategies**
- **The strategy you select for the shipping-charge rules, for the pseudocode and for the tracking states**
- **The information each strategy needs (specification, code, state model)**
- **The kinds of defects each strategy is likely and unlikely to reveal**
- **A justification for combining strategies rather than using only one**

You are expected to select and justify strategies independently for this system.

Evidence Expected
A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.

TASK 2 – Evaluate Random Testing 2 Marks
CO2 | Bloom's Level: K5 – Evaluate

A team member suggests generating 1,000 random weights and zones instead of designing test cases.

Can random testing replace systematic test design for shipCharge?

Discuss:

- **How random testing would be carried out for shipCharge**
- **The probability that random inputs hit important values such as 30.0, 30.1 and 50.0**
- **The problem of deciding the expected result (test oracle) for random inputs**

Your judgement must be supported by reasoning, not opinion.

Evidence Expected
A reasoned evaluation of the usefulness and limits of random testing for shipCharge.

TASK 3 – Design and Evaluate Equivalence Classes 6 Marks
CO2 | Bloom's Level: K5 – Evaluate

Using only the specification S1–S4, apply Equivalence Class Partitioning.

What is the smallest set of test cases that represents every valid and invalid behaviour of the shipping-charge rules?

Discuss:

- **Valid and invalid equivalence classes for weight, zone, express and parcels**
- **How you identified each class from the specification**
- **Test cases covering all valid classes with as few tests as possible**
- **Test cases covering each invalid class one at a time, and why**
- **Expected results calculated from the specification**
- **Which of your test cases would fail against the pseudocode, and what that reveals**

Simply listing classes will not receive full credit. You are expected to evaluate the effectiveness of your classes.

Evidence Expected
A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.

TASK 4 – Design and Evaluate Boundary Value Tests 4 Marks
CO2 | Bloom's Level: K5 – Evaluate

Defects in charge calculations often occur at the edges of ranges.

Which boundaries in SwiftShip are most likely to hide defects, and which form of BVA is worth its cost?

Discuss:

- **Boundary values for weight (0, 0.1, 30, 30.1, 50, 50.1), the started-kilogram rule and parcels (9, 10)**
- **Normal BVA, robust BVA and worst-case BVA, and the number of test cases each produces**
- **Test cases with expected results for the boundaries you select**
- **Which form of BVA you recommend for shipCharge, and why**

You must justify the choice of BVA form against the number of test cases and the risk.

Evidence Expected

A justified boundary value test design with expected results and a defended choice of BVA form.

TASK 5 – Apply Other Black-Box Techniques 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Express, weight and discount rules interact with each other.

Do ECP and BVA miss defects caused by combinations of rules?

Discuss:

- **A cause–effect graph or decision table for rules S3 and S4**
- **Test cases derived from the decision table**
- **Combinations that ECP/BVA did not cover, and whether they matter**

You are expected to compare the results with your ECP/BVA test set.

Evidence Expected

A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.

TASK 6 – Design and Evaluate State-Transition Tests 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

The tracking module behaves differently depending on the current state of a parcel.

Which invalid transitions could cost the company money or reputation if they were allowed?

Discuss:

- **A state-transition diagram and table for parcel tracking**
- **Test cases for all valid transitions and selected invalid transitions**
- **Tests for the "maximum two retries" rule and the cancellation rule**

You are expected to justify which invalid transitions you selected.

Evidence Expected

A state-transition model with justified valid and invalid transition tests.

TASK 7 – Construct the Control Flow Graph 4 Marks
CO2 | Bloom's Level: K5 – Evaluate

Using the pseudocode of shipCharge, construct the control flow graph.

Does the structure of the code match the structure of the specification?

Discuss:

- **Nodes and edges, with line numbers mapped to nodes**
- **Decision nodes, the loop, and the compound condition at line 1**
- **Differences between the code structure and the rules S1–S4 (for example, invalid zones)**

The graph must be derived from the code, with the mapping clearly shown.

Evidence Expected
A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.

TASK 8 – Evaluate Path Testing and Cyclomatic Complexity 4 Marks
CO2 | Bloom's Level: K5 – Evaluate

Path testing aims to exercise independent paths through the code.

How many tests are needed to exercise the logic of shipCharge, and is that number enough?

Discuss:

- **Cyclomatic complexity calculated by at least two methods**
- **A set of independent (basis) paths and test data for each path**
- **Infeasible paths, if any, and why**
- **How the loop affects path testing, and how you would test it (zero, one, many iterations)**

You must show the derivation. Stating only the final number will not receive full credit.

Evidence Expected
A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.

TASK 9 – Judge Test Adequacy and Coverage 3 Marks
CO2 | Bloom's Level: K5 – Evaluate

Management wants a single coverage number to decide whether testing is "enough".

Which coverage criterion is adequate for a module that calculates money for 40,000 parcels a day?

Discuss:

- **Statement, branch, condition and multiple-condition coverage for shipCharge**

- **The coverage achieved by your path tests from Task 8**
- **The additional tests needed to reach the next stronger criterion**
- **Whether 100 % coverage would have found the invalid-zone defect**

Your recommended criterion must be defended against cost and risk.

Evidence Expected

A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.

TASK 10 – Compare Black-Box and White-Box Effectiveness 4 Marks
CO2 | Bloom's Level: K5 – Evaluate

You now have black-box tests (Tasks 3–6) and white-box tests (Tasks 7–9).

Which approach actually found more of the important defects in shipCharge?

Discuss:

- **The defects found by black-box tests only, white-box tests only, and both**
- **Reasons why each approach missed what it missed**
- **Overlapping tests that can be removed without losing defect-finding power**
- **Whether this result would hold for other modules**

You are expected to interpret the comparison, not just count tests.

Evidence Expected

A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.

TASK 11 – Recommend and Defend the Final Test Suite 4 Marks
CO2 | Bloom's Level: K5 – Evaluate

Based on your analysis, recommend the final regression test suite for SwiftShip.

Discuss:

- **The test cases you include and the technique each comes from**
- **Why this suite is better than the alternatives**
- **The coverage and the rule combinations it achieves**
- **The number of test cases and the effort needed to run them**
- **Known gaps and risks that remain**

Your recommendation must be based on evidence from your own test design. The largest suite is not automatically the best.

Evidence Expected

A technically justified final test suite supported by black-box, white-box and coverage evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Test Suite 8 Marks
CO2 | Bloom's Level: K5 – Evaluate

Changed Requirement

SwiftShip will add a fuel surcharge of 8 % on Zone C parcels and a weekend delivery option that adds ₹50 per parcel.

The developers have changed shipCharge by adding two new decisions after line 16.

The suite must still run within 30 minutes before every release.

Re-evaluate your original solution in response to these changed conditions.

You may: RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis – 2 Marks
CO2 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed in the specification and the code**
- **Which equivalence classes, boundaries and decision-table rules are affected**
- **How the control flow graph and cyclomatic complexity change**
- **Why the 30-minute limit matters**

Evidence Expected
A clear analysis of how the changes affect the original test design.

B. Evaluation of the Original Suite – 2 Marks
CO2 | Bloom's Level: K5 – Evaluate

Evaluate whether your final test suite remains adequate.

Discuss:

- **Which existing tests are still valid, need new expected results, or are obsolete**
- **Which new paths and rule combinations are not covered**
- **Whether the coverage criterion can still be met**

Evidence Expected
A technically supported evaluation of the original suite under the new conditions.

C. Adaptation or Redesign – 2 Marks
CO2 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the test suite.

If you modify or redesign it:

Discuss:

- **Describe the new and changed test cases**
- **Explain which techniques you used to derive them**
- **Show the effect on coverage and the number of test cases**

If you retain the original suite, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate test-design response to the changed requirement with clear reasoning.

D. Evidence-Based Defence – 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defend your final test suite using evidence.

Use:

Discuss:

- **Equivalence classes and boundaries**
- **Decision-table combinations**
- **Cyclomatic complexity and basis paths**
- **Coverage achieved**
- **Execution time limit**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final test suite integrating test design evidence and the changed constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Select Test Case Design Strategies	5
Task 2 – Evaluate Random Testing	2
Task 3 – Design and Evaluate Equivalence Classes	6
Task 4 – Design and Evaluate Boundary Value Tests	4
Task 5 – Apply Other Black-Box Techniques	3
Task 6 – Design and Evaluate State-Transition Tests	3
Task 7 – Construct the Control Flow Graph	4
Task 8 – Evaluate Path Testing and Cyclomatic Complexity	4
Task 9 – Judge Test Adequacy and Coverage	3
Task 10 – Compare Black-Box and White-Box Effectiveness	4

Task	Marks
Task 11 – Recommend and Defend the Final Test Suite	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	50

SOFTWARE TESTING (CSA37)
ASSIGNMENT 2 – CO2

CO2 Coverage: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator (Total Marks - 50)

Duration:1week

Scenario — Courier Shipping-Charge Calculator (SwiftShip)

A courier company is launching SwiftShip, an online service that calculates the shipping charge for a parcel and tracks it until delivery. More than 40,000 parcels are booked every day. An undercharge causes revenue loss; an overcharge causes customer complaints and refunds.

You are the Test Designer. You must design test cases, evaluate their effectiveness and defend the final test suite that will be used before every release.

Shipping-charge rules (specification extract):

Rule	Specification
S1	Weight (kg, up to one decimal place) must be greater than 0 and not more than 50. Otherwise the booking is rejected as INVALID.
S2	Charge is calculated for every started kilogram: Zone A ₹40/kg, Zone B ₹60/kg, Zone C ₹90/kg. Any other zone is INVALID.
S3	Express delivery adds 50 % to the charge. Express is allowed only up to 30 kg; above 30 kg the request is NOT_ALLOWED.
S4	Customers who booked 10 or more parcels in the current month receive a 10 % discount on the final charge.

Shipping-charge module – Pseudocode under test:

```
function shipCharge(weight, zone, express, parcels)
1   if weight <= 0 or weight > 50 then
2       return INVALID
3   rate = 40
4   if zone == "B" then
5       rate = 60
6   else if zone == "C" then
7       rate = 90
8   charge = 0
9   w = weight
10  while w > 0 do
11      charge = charge + rate
```

```
12      w = w - 1
13  if express then
14      if weight > 30 then
15          return NOT_ALLOWED
16      charge = charge * 1.5
17  if parcels >= 10 then
18      charge = charge * 0.9
19  return charge
```

Parcel tracking states:

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED.

A parcel can move from OUT_FOR_DELIVERY to FAILED_ATTEMPT, and from FAILED_ATTEMPT back to OUT_FOR_DELIVERY (maximum two retries) or to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Design Challenge:

SwiftShip will add a fuel surcharge and weekend delivery, and the shipping-charge code will be modified.

Re-evaluate your test suite and determine whether it should be retained, modified or redesigned.

TASK 1 – Select Test Case Design Strategies 5 Marks
CO2 | Bloom's Level: K5 – Evaluate

SwiftShip has a calculation module (shipCharge) and a tracking module.

Which test design strategy will find the most important defects in each module with the least effort?

Your answer must include:

Discuss:

- **The difference between black-box and white-box test design strategies**
- **The strategy you select for the shipping-charge rules, for the pseudocode and for the tracking states**
- **The information each strategy needs (specification, code, state model)**
- **The kinds of defects each strategy is likely and unlikely to reveal**
- **A justification for combining strategies rather than using only one**

You are expected to select and justify strategies independently for this system.

Evidence Expected
A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.

Ans:

Choosing Test Design Strategies for SwiftShip

My aim is to choose, for each part of SwiftShip, the strategy that finds the most important defects with the least effort. I select black-box techniques (equivalence classes, boundary values and a decision table) for the charge rules S1–S4, white-box control-flow and path testing for the shipCharge pseudocode, and state-transition testing for tracking. No single strategy is enough, because each one can only see one kind of defect.

Table 1: Black-box and white-box test design compared (applied to SwiftShip)

Aspect	Black-box design	White-box design
Test derived from	The specification S1–S4 and the state model; the code is not needed	The structure of the code: statements, decisions, loops and paths of shipCharge
Question it asks	"Does SwiftShip do what the rules say?"	"Has every part of the code been executed, and does it behave as written?"
Needs	Specification, state model, business rules	Source code or pseudocode and its control flow graph
Good at finding	Missing or wrong rules, wrong limits, wrong expected values, illegal state changes	Untested decisions, wrong operators, loop errors, dead or infeasible paths
Weak at finding	Code that spec-based inputs never exercise	Missing code (omissions): something the specification requires but the code does not contain
SwiftShip example	A zone "D" booking is accepted at ₹40/kg: S2 says INVALID but the code has no branch for it	A basis path that no specification-driven test happened to follow

Table 2: Strategy selected for each part of SwiftShip

Part	Strategy selected	Information needed	Defects it is likely to reveal	Defects it is unlikely to reveal	Effort
Charge rules S1–S4	Equivalence class partitioning, boundary value analysis, decision table (black-box)	Specification S1–S4 only; can be done before the code exists	Wrong weight limits, invalid zone accepted, started-kilogram error, express limit 30/30.1, discount threshold 9/10, rule interactions	Structural faults such as an unreachable branch or a loop that misbehaves for values the spec does not suggest	About 20 tests cover every rule
shipCharge pseudocode	Control flow graph, basis-path testing, branch and condition coverage (white-box)	The 19-line pseudocode and its CFG	Decisions never exercised, wrong comparison operators, loop errors, infeasible paths	A rule that is missing from the code (invalid zone)	V(G) = 8, so 8 basis paths
Tracking states	State-transition testing (black-box, model based)	The state model: 8 states, 8 valid transitions, retry and cancel rules	Illegal transitions, retry limit, cancellation window, skipped states	Arithmetic or pricing faults; states missing from the model	13 short sequences

Why the strategies must be combined

- **Each technique has a blind spot, and the blind spots are different.** The one real defect in shipCharge (an unknown zone is silently charged at ₹40/kg) is an omission. Statement

and branch coverage of 100% can be reached using only zones A, B and C (Task 9), so white-box testing alone would never reveal it. Only an invalid-zone test derived from S2 does.

- **Black-box tests do not prove the code has been exercised.** Spec-driven inputs can leave a decision outcome untouched, and they cannot tell the team that the zero-iteration path of the loop is infeasible. The CFG and basis paths provide that structural check.
- **The tracking module has a different shape.** It is a state machine rather than a formula, so the rules (retry limit, cancellation window) are only visible through a state model.
- **Exhaustive testing is not an option.** shipCharge alone has 500 valid weights × 3 zones × 2 express values × even 100 parcel counts = 300,000 valid combinations. A designed suite of about 30 tests is what the 30-minute budget needs.

Conclusion: the strongest and cheapest arrangement is black-box first (cheap, available before coding, finds omissions), white-box second (measures what the black-box tests missed and finds dead paths), with state-transition tests for the tracking module.

TASK 2 – Evaluate Random Testing 2 Marks
CO2 | Bloom's Level: K5 – Evaluate

A team member suggests generating 1,000 random weights and zones instead of designing test cases.

Can random testing replace systematic test design for shipCharge?

Discuss:

- **How random testing would be carried out for shipCharge**
- **The probability that random inputs hit important values such as 30.0, 30.1 and 50.0**
- **The problem of deciding the expected result (test oracle) for random inputs**

Your judgement must be supported by reasoning, not opinion.

Evidence Expected
A reasoned evaluation of the usefulness and limits of random testing for shipCharge.

Ans:

Can Random Testing Replace Systematic Test Design?

My position is that random testing cannot replace systematic design for shipCharge. It is a cheap supplement, but it hits the important boundary values only by luck, it never goes beyond what its generator was told to generate, and it cannot say what the correct answer is.

How random testing would be carried out

A script draws 1,000 values of weight (for example uniformly from −10.0 to 100.0 in steps of 0.1), a zone from {A, B, C}, a random express flag and a parcel count from 0 to 30, calls shipCharge, and compares each result with an expected value. It needs no analysis of the specification, which is its attraction.

Probability of hitting the important values

With 1,000 independent uniform draws, the chance that a specific weight appears at least once is 1 − (1 − 1/N)^1000, where N is the number of possible weights. Treating weights as one-decimal values:

Table 3: Chance that 1,000 random tests reach the key boundary values

Target	Generator range	N (possible values)	Chance one named value is hit at least once	Chance all of 30.0, 30.1 and 50.0 are hit
Weights 30.0, 30.1, 50.0	Valid weights only (0.1 to 50.0)	500	1 − (499/500)^1000 = 86.5%	0.865^3 = 64.7%
Weights 30.0, 30.1, 50.0	−10.0 to 100.0	1,101	59.7%	0.597^3 = 21.3%
30.1 together with express = Yes	Valid weights, express chosen at random	500 × 2	63.2%	—
30.1 + express + zone C + parcels ≥ 10	Valid weights, random flags	500 × 2 × 3 × 2	15.4%	—
Weight 50.1 or 0 (the invalid sides)	Valid weights only	0 (never generated)	0%	—

Even in the best case (the generator already knows the valid range), roughly one run in three misses at least one of the three key weights, and a defect at exactly 30.1 with express would be missed in 37% of runs. A designed boundary test makes the probability 100% with three tests. If the generator draws real numbers instead of one-decimal values, the probability of hitting 30.1 exactly is 0. Likewise, a generator that draws zones only from {A, B, C} can never produce the invalid-zone case, so the D1 defect would not be found. Random testing explores only the world its generator imagines.

The oracle problem

Every random input needs an expected result. There are three possible sources, and each has a weakness:

- Calculating 1,000 answers by hand is impractical.
- A second program written from S1–S4 (a reference model) can calculate them, but it has to be built and tested itself, and if it repeats the same misreading of the specification, the same wrong answer is recorded as expected. Building it costs about as much as writing the systematic tests.
- Property checks (express charge = 1.5 × the non-express charge; discount = 0.9 × the undiscounted charge; the charge never decreases as weight increases) cost almost nothing but cannot detect a wrong rate such as zone B at ₹50 instead of ₹60.

Table 4: Random testing compared with systematic design for shipCharge

Criterion	Random (1,000 tests)	Systematic design (ECP + BVA + decision table)
Reaches boundaries 30.0 / 30.1 / 50.0	About 65% of runs at best	Always, by construction

Criterion	Random (1,000 tests)	Systematic design (ECP + BVA + decision table)
Reaches invalid zone, weight 0 or 50.1	Only if the generator is told to	Always (Tasks 3–4)
Expected results	Need a reference model or properties	Calculated once from the specification
Cost to run	Seconds	Seconds
Cost to build	Generator plus oracle	Analysis of the specification
Best use	Finding crashes and large calculation errors in bulk; differential check against the reference model	Proving that every rule and boundary is correct

Judgement: keep random testing as a nightly supplement (crash detection and comparison with a reference model) but never as a substitute for the designed suite.

TASK 3 – Design and Evaluate Equivalence Classes 6 Marks
CO2 | Bloom's Level: K5 – Evaluate

Using only the specification S1–S4, apply Equivalence Class Partitioning.

What is the smallest set of test cases that represents every valid and invalid behaviour of the shipping-charge rules?

Discuss:

- **Valid and invalid equivalence classes for weight, zone, express and parcels**
- **How you identified each class from the specification**
- **Test cases covering all valid classes with as few tests as possible**
- **Test cases covering each invalid class one at a time, and why**
- **Expected results calculated from the specification**
- **Which of your test cases would fail against the pseudocode, and what that reveals**

Simply listing classes will not receive full credit. You are expected to evaluate the effectiveness of your classes.

Evidence Expected

A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.

Ans:

Equivalence Class Partitioning for the Charge Rules

I derive the classes from the wording of S1–S4 only, one class per distinct behaviour the specification describes. I then build the smallest set of tests that covers every valid class together and every invalid class separately, calculate the expected results from the specification, and run the tests against the pseudocode to judge how well the classes expose defects.

Table 5: Equivalence classes identified from S1–S4

ID	Input	Class	Rule that defines it	Valid / Invalid	Expected behaviour
W1	Weight	0 < weight ≤ 50 (one decimal)	S1	Valid	Charge calculated
W2	Weight	weight ≤ 0	S1	Invalid	INVALID
W3	Weight	weight > 50	S1	Invalid	INVALID
Z1	Zone	A	S2	Valid	₹40 per started kg
Z2	Zone	B	S2	Valid	₹60 per started kg
Z3	Zone	C	S2	Valid	₹90 per started kg
Z4	Zone	Any other value (for example D)	S2: "any other zone is INVALID"	Invalid	INVALID
E1	Express	No	S3	Valid	No increase
E2	Express	Yes and weight ≤ 30	S3	Valid	Charge × 1.5
E3	Express	Yes and weight > 30	S3	Invalid	NOT_ALLOWED
P1	Parcels this month	0 to 9	S4	Valid	No discount
P2	Parcels this month	10 or more	S4	Valid	Charge × 0.9

How the classes were identified

- Each numbered rule gives a condition and the result for each side of that condition: S1 splits weight into one valid and two invalid classes, S2 splits zone into three valid zones and "any other", S3 splits express into three, and S4 into two.
- E2 and E3 depend on two inputs (express and weight), so they are classes of a combination, not of a single input.
- The specification is silent about some inputs: a weight with more than one decimal place, a negative parcel count and a lower-case zone. I did not invent classes for them; I record them as specification gaps to be clarified (Task 11).

Test cases for the valid classes (as few as possible)

The largest number of valid classes for one input is 3 (the zones), so at least 3 tests are needed, and each test can cover one zone together with one class of every other input.

Table 6: Valid-class tests and expected results

Test	Weight	Zone	Express	Parcels	Classes covered	Calculation from the specification	Expected
T1	10.0	A	No	3	W1 Z1 E1 P1	10 kg × ₹40 = 400; no express; no discount	₹400
T2	12.5	B	Yes	12	W1 Z2 E2 P2	13 started kg × ₹60 = 780; × 1.5 = 1,170; × 0.9 = 1,053	₹1,053
T3	25.3	C	No	10	W1 Z3 E1 P2	26 started kg × ₹90 = 2,340; × 0.9 = 2,106	₹2,106

Test cases for the invalid classes (one at a time)

Table 7: Invalid-class tests, each with exactly one invalid input

Test	Weight	Zone	Express	Parcels	Class covered	Expected
T4	−5	A	No	3	W2	INVALID
T5	60	A	No	3	W3	INVALID
T6	10.0	D	No	3	Z4	INVALID
T7	35.0	B	Yes	5	E3	NOT_ALLOWED

Each invalid class gets its own test because a program normally stops at the first check it fails. If one test combined weight 60 with zone D, line 2 would return INVALID and the zone check would never run, so the zone defect would be masked. Seven tests (3 valid + 4 invalid) cover all 12 classes.

Which tests fail against the pseudocode

Table 8: Result of running the seven tests on the pseudocode

Test	Expected	Result of the pseudocode	Verdict
T1	₹400	400	Pass
T2	₹1,053	1,053	Pass
T3	₹2,106	2,106	Pass
T4	INVALID	INVALID (line 2)	Pass
T5	INVALID	INVALID (line 2)	Pass
T6	**INVALID**	**400** (zone D falls through to the default rate of 40)	**FAIL**
T7	NOT_ALLOWED	NOT_ALLOWED (line 15)	Pass

What the result reveals

- **Defect D1:** the code tests only for “B” and “C” and treats every other zone as zone A (rate = 40 at line 3). S2 requires INVALID. A booking to zone “D” is accepted at the cheapest rate, which causes silent undercharging.
- **Effectiveness:** the classes found 1 of 1 specification-level defects with 7 tests, and 12 of 12 classes are covered. The three valid tests alone pass and would have given false confidence; the defect appears only through an invalid class, which shows why invalid classes must be tested explicitly.
- **Limitation:** ECP assumes every value in a class behaves alike, so it does not test the edges of W1 and E2 (0.1, 30.0, 30.1, 50.0). A comparison error such as “weight ≥ 30” would pass all seven tests. This is the gap that Task 4 closes.

TASK 4 – Design and Evaluate Boundary Value Tests 4 Marks
CO2 | Bloom's Level: K5 – Evaluate

Defects in charge calculations often occur at the edges of ranges.

Which boundaries in SwiftShip are most likely to hide defects, and which form of BVA is worth its cost?

Discuss:

- **Boundary values for weight (0, 0.1, 30, 30.1, 50, 50.1), the started-kilogram rule and parcels (9, 10)**
- **Normal BVA, robust BVA and worst-case BVA, and the number of test cases each produces**
- **Test cases with expected results for the boundaries you select**
- **Which form of BVA you recommend for shipCharge, and why**

You must justify the choice of BVA form against the number of test cases and the risk.

Evidence Expected

A justified boundary value test design with expected results and a defended choice of BVA form.

Ans:

Boundary Value Analysis for shipCharge

The edges of ranges are where comparison operators (<, ≤, >, ≥) are written wrongly, so a money calculation deserves tests exactly at each edge. I list the boundaries from S1–S4 and the code, compare the three BVA forms by number of tests and risk, and select the form that gives the most protection for the least cost.

Table 9: Boundaries of shipCharge and their expected results (zone A, no express, 5 parcels unless stated)

Boundary	Rule	Value tested	Expected result and calculation
Lower weight limit	S1	0	INVALID (weight must be greater than 0)
Just above lower limit	S1, S2	0.1	1 started kg × ₹40 = ₹40
Started-kilogram edge	S2	1.0	Exactly 1 kg: ₹40
Just over a kilogram	S2	1.1	2 started kg × ₹40 = ₹80
Express limit (allowed)	S3	30.0, express	30 kg × ₹40 = 1,200; × 1.5 = ₹1,800
Express limit (rejected)	S3	30.1, express	NOT_ALLOWED
Same weight without express	S3	30.1, no express	31 kg × ₹40 = ₹1,240 (express rule must not affect it)
Upper weight limit	S1	50.0	50 kg × ₹40 = ₹2,000
Just above upper limit	S1	50.1	INVALID
Discount threshold (below)	S4	Parcels = 9, weight 10	10 kg × ₹40 = ₹400; no discount
Discount threshold (at)	S4	Parcels = 10, weight 10	400 × 0.9 = ₹360

The three forms of BVA and the number of tests

BVA applies to ordered numeric inputs. For shipCharge these are weight and parcels (n = 2); zone and express are categories and are handled by ECP and the decision table.

Table 10: Number of tests produced by each form of BVA

Form	Values used for each variable	Formula	Tests for n = 2 (weight, parcels)	If zone and express were also treated as variables (n = 4)	What it adds
Normal BVA (single fault)	min, min+, nominal, max−, max	4n + 1	9	17	Faults at the valid edges
Robust BVA	Normal values plus min− and max+	6n + 1	13	25	Also the invalid side of each edge (0 and 50.1)
Worst-case BVA	All combinations of the five normal values	5^n	25	625	Interactions between edges of different inputs
Robust worst-case BVA	All combinations of seven values	7^n	49	2,401	Interactions including invalid edges

The BVA tests I select

Table 11: Selected boundary tests with expected results and the result on the pseudocode

Test	Weight	Zone	Express	Parcels	Expected	Result on the pseudocode
B1	0	A	No	5	INVALID	INVALID – Pass
B2	0.1	A	No	5	₹40	40 – Pass
B3	1.0	A	No	5	₹40	40 – Pass
B4	1.1	A	No	5	₹80	80 – Pass
B5	50.0	A	No	5	₹2,000	2000 – Pass
B6	50.1	A	No	5	INVALID	INVALID – Pass
B7	30.0	A	Yes	5	₹1,800	1800 – Pass
B8	30.1	A	Yes	5	NOT_ALLOWED	NOT_ALLOWED – Pass
B9	30.1	A	No	5	₹1,240	1240 – Pass
B10	10.0	A	No	9	₹400	400 – Pass
B11	10.0	A	No	10	₹360	360 – Pass

Which form is worth its cost

- **Recommendation: robust BVA applied to the specific edges (11 tests).** Robust values matter because S1 explicitly rejects 0 and 50.1, so the invalid side of the edge is a rule to be tested (B1, B6). The express limit (B7–B9) and the started-kilogram edge (B3–B4) are not range limits of an input but are boundaries of rules, and are included because the money result changes exactly there.

- **Why not worst-case (25 tests) or robust worst-case (49):** the extra tests mostly combine edges of different inputs, for example weight 50.0 with parcels 10. The rule combinations that can actually interact (express, weight above 30, discount) are covered more cheaply by the decision table in Task 5, with 8 tests instead of 25.
- **Risk against cost:** each test costs seconds to run automatically, but a wrong operator at 30, 50 or 10 misprices every parcel on that edge among 40,000 booked each day. The cost of 11 tests is therefore small compared with the exposure.
- **Result:** all 11 tests pass on the pseudocode, so BVA found no defect here; the comparison operators at lines 1, 14 and 17 and the loop are correct (I also ran the loop for all 500 one-decimal weights and it always agrees with "started kilograms"). BVA still earns its place as a regression guard: in Task 10 these 11 tests caught six of the ten seeded faults, and two of them ("weight ≥ 50" and "weight ≥ 30") were caught by no other test set.

TASK 5 – Apply Other Black-Box Techniques 3 Marks
CO2 | Bloom's Level: K5 – Evaluate

Express, weight and discount rules interact with each other.

Do ECP and BVA miss defects caused by combinations of rules?

Discuss:

- **A cause–effect graph or decision table for rules S3 and S4**
- **Test cases derived from the decision table**
- **Combinations that ECP/BVA did not cover, and whether they matter**

You are expected to compare the results with your ECP/BVA test set.

Evidence Expected
A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.

Ans:

Do ECP and BVA Miss Defects Caused by Combinations of Rules?

Yes, they can. ECP and BVA vary one input at a time while the other inputs are held at ordinary values. Rules S3 and S4 interact: whether a discount is applied, and to what amount, depends on whether express was applied first. I build a decision table for S3 and S4 and compare its rules with the ECP and BVA tests from Tasks 3 and 4.

Decision table for rules S3 and S4

Conditions: C1 = express requested; C2 = weight > 30; C3 = parcels ≥ 10. Actions: A1 = charge × 1.5; A2 = NOT_ALLOWED; A3 = charge × 0.9. Three binary conditions give 2^3 = 8 rules.

Table 12: Decision table for S3 and S4 with a test derived from each rule

Rule	C1 express	C2 weight > 30	C3 parcels ≥ 10	Action	Test derived (weight, zone, express, parcels)	Expected
R1	No	No	No	Base charge	10, A, No, 5	₹400

Rule	C1 express	C2 weight > 30	C3 parcels ≥ 10	Action	Test derived (weight, zone, express, parcels)	Expected
R2	No	No	Yes	A3	10, A, No, 10	₹360
R3	No	Yes	No	Base charge	35, B, No, 5	35 × 60 = ₹2,100
R4	No	Yes	Yes	A3	35, B, No, 12	2,100 × 0.9 = ₹1,890
R5	Yes	No	No	A1	10, C, Yes, 5	900 × 1.5 = ₹1,350
R6	Yes	No	Yes	A1, A3	10, C, Yes, 12	1,350 × 0.9 = ₹1,215
R7	Yes	Yes	No	A2	35, A, Yes, 5	NOT_ALLOWED
R8	Yes	Yes	Yes	A2 (no discount, no charge)	35, A, Yes, 12	NOT_ALLOWED

R7 and R8 behave identically (parcels does not matter once express is rejected), so there are 7 distinct outcomes, but I keep both tests because R8 checks that a discount does not change a rejection. All eight tests pass on the pseudocode.

Comparison with the ECP and BVA tests

Table 13: Which decision-table rules the ECP tests (T1–T7) and BVA tests (B1–B11) already cover

Rule	Covered by ECP?	Covered by BVA?	New test needed?
R1	Yes (T1)	Yes (B10)	No
R2	Yes (T3, non-express, parcels 10)	Yes (B11)	No
R3	No	Yes (B9: 30.1 kg, no express)	No
R4	**No**	**No**	**Yes (R4 test)**
R5	No (T2 has 12 parcels)	Yes (B7)	No
R6	Yes (T2)	No	No
R7	Yes (T7)	Yes (B8)	No
R8	**No**	**No**	**Yes (R8 test)**

- ECP and BVA together reach 6 of the 8 rules (75%). The two they miss are R4 (a heavy parcel without express, with a discount) and R8 (a discount requested together with a rejected express).
- **Do the missed combinations matter?** R4 matters: code that nests the discount inside a weight check (for example "if weight ≤ 30 then apply the discount") would pass every ECP and BVA test and still overcharge every heavy, loyal customer; this fault type is realistic because line 14 already branches on weight. R8 matters less in the current code, because line 15 returns before line 17, but it protects against the discount being moved before the express check during later changes.
- **Combinations I deliberately do not test:** the 3 zones are not crossed with every rule, because the zone only selects the rate at lines 3–7 and the decision structure after line 8

never reads it again. This argument stops being valid in Task 12, when the fuel surcharge reads the zone a second time.

TASK 6 – Design and Evaluate State-Transition Tests 3 Marks
CO2 | Bloom's Level: K5 – Evaluate

The tracking module behaves differently depending on the current state of a parcel.

Which invalid transitions could cost the company money or reputation if they were allowed?

Discuss:

- **A state-transition diagram and table for parcel tracking**
- **Test cases for all valid transitions and selected invalid transitions**
- **Tests for the "maximum two retries" rule and the cancellation rule**

You are expected to justify which invalid transitions you selected.

Evidence Expected
A state-transition model with justified valid and invalid transition tests.

Ans:

State-Transition Testing of Parcel Tracking

The tracking module has 8 states and 8 allowed transitions. Of the 64 possible state–event pairs, 56 are illegal. The tests must show that the 8 legal transitions work and that the illegal ones that could cost money or reputation are refused.

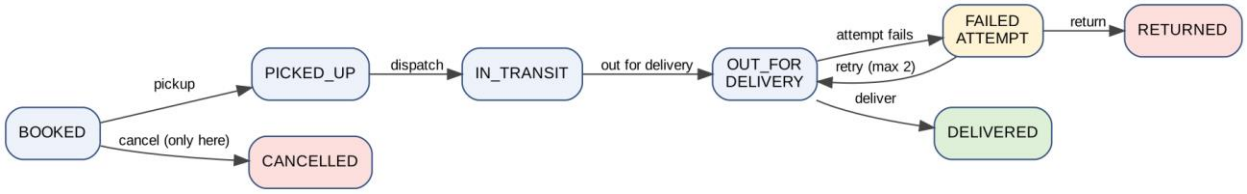


Figure 1: State-transition diagram of parcel tracking (red states are terminal; FAILED_ATTEMPT is the retry loop)

Table 14: State-transition table (rows are states; "–" means the event must be rejected)

State	pickup	dispatch	out for delivery	deliver	attempt fails	retry	return	cancel
BOOKED	PICKED_UP	–	–	–	–	–	–	CANCELLED
PICKED_UP	–	IN_TRANSIT	–	–	–	–	–	–
IN_TRANSIT	–	–	OUT_FOR_DELIVERY	–	–	–	–	–
OUT_FOR_DELIVERY	–	–	–	DELIVERED	FAILED_ATTEMPT	–	–	–
FAILED_ATTEMPT	–	–	–	–	–	OUT_FOR_DELIVERY (only if fewer than 2 retries used)	RETURNED	–
DELIVERED	–	–	–	–	–	–	–	–
RETURNED	–	–	–	–	–	–	–	–
CANCELLED	–	–	–	–	–	–	–	–

I read “maximum two retries” as: one first attempt plus at most two retries, so a parcel can fail at most three times; after the third failure only RETURNED is allowed. I read the cancellation rule as: allowed only in BOOKED.

Table 15: Tracking test cases (13 tests)

Test	Type	Sequence of events	Expected result
ST1	Valid	BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED	All four transitions accepted
ST2	Valid	OUT_FOR_DELIVERY → FAILED_ATTEMPT → OUT_FOR_DELIVERY (retry 1) → DELIVERED	Failure, retry and delivery accepted
ST3	Valid	OUT_FOR_DELIVERY → FAILED_ATTEMPT → RETURNED	Return accepted
ST4	Valid	BOOKED → CANCELLED	Cancellation accepted
ST5	Retry limit	Fail, retry 1, fail, retry 2, then DELIVERED	Both retries accepted; delivery accepted
ST6	Retry limit (invalid)	Fail, retry 1, fail, retry 2, fail (third failure), then a third retry	Third retry rejected; state stays FAILED_ATTEMPT
ST7	Retry limit	After the third failure, return	RETURNED accepted
X1	Cancellation (invalid)	PICKED_UP → cancel	Rejected; state stays PICKED_UP
X2	Cancellation (invalid)	OUT_FOR_DELIVERY → cancel	Rejected; state stays OUT_FOR_DELIVERY
X3	Terminal state (invalid)	DELIVERED → cancel	Rejected; no refund can be triggered
X4	Skipped state (invalid)	BOOKED → deliver	Rejected; state stays BOOKED
X5	Terminal state (invalid)	RETURNED → out for delivery	Rejected; state stays RETURNED
X6	Terminal state (invalid)	CANCELLED → pickup	Rejected; state stays CANCELLED

ST1 to ST4 together execute all 8 valid transitions (8 of 8 covered). ST5 to ST7 test the boundary of the retry rule from both sides: the second retry is allowed (ST5) and the third is not (ST6).

Why these invalid transitions were selected

Table 16: Selected invalid transitions and the loss each would cause

Test	Invalid transition	Loss if the system allowed it
X1, X2	Cancel after PICKED_UP or while OUT_FOR_DELIVERY	Refund for a parcel the courier already holds or is delivering: direct revenue loss and a parcel in limbo
X3	DELIVERED → cancel	Refund after successful delivery: money loss and a false record
X4	BOOKED → deliver (skipping pickup and transit)	A false “delivered” message to a customer whose parcel is still at home: reputation damage and disputes

Test	Invalid transition	Loss if the system allowed it
X5	RETURNED → out for delivery	A returned parcel re-enters delivery: double handling cost, conflicting information
X6	CANCELLED → pickup	Courier collects a cancelled and unpaid booking: wasted trip

The selection rule was: test an invalid transition when it leaks money (X1–X3), gives the customer false information (X4), or revives a finished parcel (X5, X6). One test per terminal state is enough, because all three terminal states have the same behaviour (accept nothing). I did not select the other 50 or so invalid pairs, such as IN_TRANSIT → PICKED_UP, whose consequence is only a wrong status label that staff can correct.

TASK 7 – Construct the Control Flow Graph 4 Marks
CO2 | Bloom's Level: K5 – Evaluate

Using the pseudocode of shipCharge, construct the control flow graph.

Does the structure of the code match the structure of the specification?

Discuss:

- **Nodes and edges, with line numbers mapped to nodes**
- **Decision nodes, the loop, and the compound condition at line 1**
- **Differences between the code structure and the rules S1–S4 (for example, invalid zones)**

The graph must be derived from the code, with the mapping clearly shown.

Evidence Expected
A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.

Ans:

Control Flow Graph of shipCharge

I built the graph from the 19 lines of pseudocode. Consecutive statements that always execute together are combined into one node; every decision has its own node; a single exit node is added so that the three return statements (lines 2, 15 and 19) all lead to the same end. The result is 18 nodes and 24 edges.

Table 17: Node–line mapping

Node	Line(s)	Statement	Kind
N1	1	weight <= 0 or weight > 50	Decision (compound condition)
N2	2	return INVALID	Return
N3	3	rate = 40	Process
N4	4	zone == "B"	Decision
N5	5	rate = 60	Process

Node	Line(s)	Statement	Kind
N6	6	zone == "C"	Decision
N7	7	rate = 90	Process
N8	8–9	charge = 0; w = weight	Process
N9	10	while w > 0	Decision (loop)
N10	11–12	charge = charge + rate; w = w − 1	Process (loop body)
N11	13	if express	Decision
N12	14	weight > 30	Decision
N13	15	return NOT_ALLOWED	Return
N14	16	charge = charge × 1.5	Process
N15	17	parcels >= 10	Decision
N16	18	charge = charge × 0.9	Process
N17	19	return charge	Return
N18	—	Exit node	Exit

Edges (24): N1→N2 (T), N1→N3 (F), N3→N4, N4→N5 (T), N4→N6 (F), N5→N8, N6→N7 (T), N6→N8 (F), N7→N8, N8→N9, N9→N10 (T), N10→N9, N9→N11 (F), N11→N12 (T), N11→N15 (F), N12→N13 (T), N12→N14 (F), N14→N15, N15→N16 (T), N15→N17 (F), N16→N17, N2→N18, N13→N18, N17→N18.

The compound condition at line 1 (“weight <= 0 or weight > 50”) is drawn as one decision node, because it makes one branch decision. At condition level it is two nodes (weight <= 0, then weight > 50); this matters for condition coverage in Task 9 and raises the node and edge counts to 19 and 26.

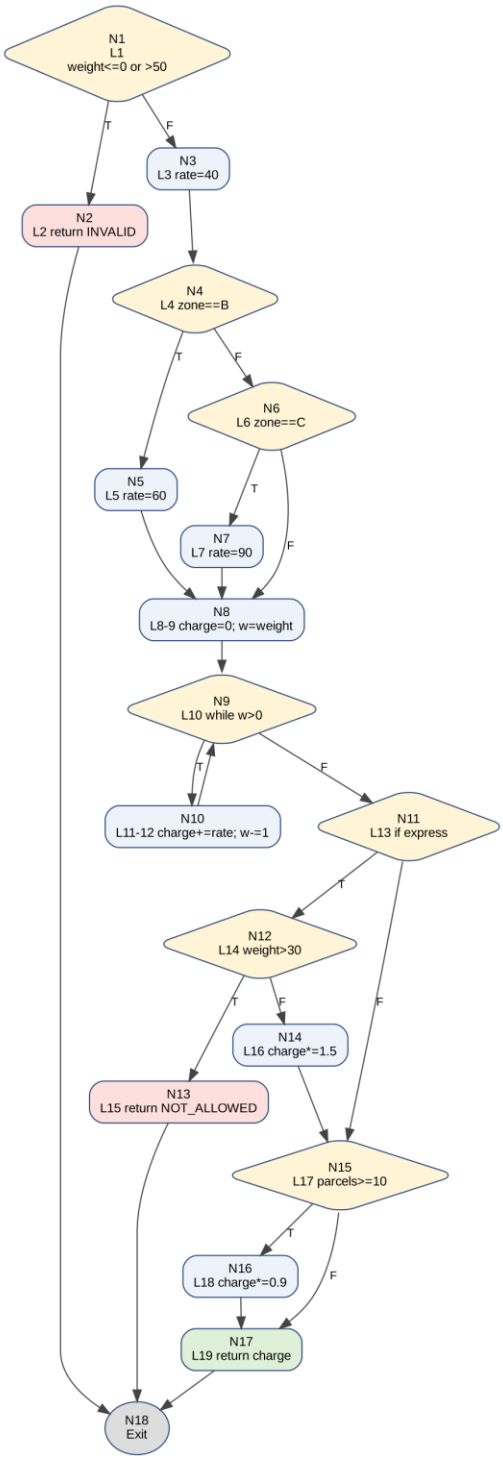


Figure 2: Control flow graph of shipCharge (diamonds are decisions; red nodes are early returns)

Does the code structure match the specification?

Table 18: Specification structure compared with the structure of the graph

Rule	What the specification requires	What the graph shows	Match?
S1	Reject weight ≤ 0 and weight > 50; weight has at most one decimal	N1 rejects both ends in one compound decision. There is no node that checks the number of decimals	Mostly; decimal rule not implemented
S2 (rates)	Zone A ₹40, B ₹60, C ₹90 per started kilogram	N3–N7 select the rate; the loop N9–N10 adds the rate once per started kilogram	Yes
S2 (other zones)	**Any other zone is INVALID**	**N4 and N6 have only B, C and "everything else". No edge leads from the zone decisions to N2 (INVALID). Zone D flows to N8 with the rate of N3**	**No – missing branch (defect D1)**
S3	Express adds 50%; rejected above 30 kg	N11 → N12 → N13 / N14 gives the allowed and the NOT_ALLOWED outcome	Yes
S4	10% off the final charge when parcels ≥ 10	N15 → N16 occurs after the express factor, so the discount applies to the final charge	Yes
Order of rules	Weight check first, then charge, express, discount	Same order: N1, N3–N8, loop, N11–N14, N15–N16	Yes

Evaluation: the code follows the structure of S1, S3 and S4, but it does not follow S2. The specification has four outcomes for zone (A, B, C, invalid) while the graph has three, and the missing outcome is exactly the failure of test T6. This mismatch is visible by comparing the graph with the specification, without running anything. The loop is a further structural difference: the specification says "every started kilogram" and the code computes this by repeated subtraction, which is equivalent for one-decimal weights up to 50.

TASK 8 – Evaluate Path Testing and Cyclomatic Complexity 4 Marks
CO2 | Bloom's Level: K5 – Evaluate

Path testing aims to exercise independent paths through the code.

How many tests are needed to exercise the logic of shipCharge, and is that number enough?

Discuss:

- **Cyclomatic complexity calculated by at least two methods**
- **A set of independent (basis) paths and test data for each path**
- **Infeasible paths, if any, and why**
- **How the loop affects path testing, and how you would test it (zero, one, many iterations)**

You must show the derivation. Stating only the final number will not receive full credit.

Evidence Expected

A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.

Ans:

Cyclomatic Complexity, Basis Paths and Loop Testing

Cyclomatic complexity V(G) gives the number of linearly independent paths through the graph, which is the smallest number of tests that exercise every decision outcome once. I calculate it in three ways from Figure 2, derive a basis set of paths with test data, and then check whether that number of tests is enough.

Table 19: Cyclomatic complexity of shipCharge by three methods

Method	Calculation	V(G)
1. Edges and nodes: V = E − N + 2	24 − 18 + 2	8
2. Predicate nodes + 1	Predicates: N1, N4, N6, N9, N11, N12, N15 = 7; 7 + 1	8
3. Regions of the planar graph	7 bounded regions (invalid-weight bypass, zone B, zone C, loop, express block, NOT_ALLOWED bypass, discount) + 1 outer region	8
Condition level (line 1 split into two conditions)	8 predicates + 1 (E = 26, N = 19: 26 − 19 + 2 = 9)	9

Table 20: Basis paths and test data (V(G) = 8 paths)

Path	Nodes traversed	New edge(s) that make it independent	Test (weight, zone, express, parcels)	Expected
P0 (baseline)	N1, N3, N4, N6, N8, N9, N10, N9, N11, N15, N17, N18	—	1.0, A, No, 5	₹40
P1	N1, N2, N18	N1→N2	0, A, No, 5	INVALID
P2	…N4, N5, N8, …	N4→N5, N5→N8	1.0, B, No, 5	₹60
P3	…N4, N6, N7, N8, …	N6→N7, N7→N8	1.0, C, No, 5	₹90
P4	…N8, N9, N10, N9, N10, N9, N10, N9, N11, …	Loop traversed several times	2.5, A, No, 5 (3 started kg)	₹120
P5	…N9, N11, N12, N13, N18	N11→N12, N12→N13, N13→N18	35, A, Yes, 5	NOT_ALLOWED
P6	…N11, N12, N14, N15, N17, N18	N12→N14, N14→N15	10, A, Yes, 5	400 × 1.5 = ₹600
P7	…N11, N15, N16, N17, N18	N15→N16, N16→N17	1.0, A, No, 10	40 × 0.9 = ₹36

Infeasible paths

- **Zero iterations of the loop (N8 → N9 → N11 on the first test of N9) is infeasible.** Line 1 guarantees weight > 0, so "w > 0" is always true the first time it is tested. Therefore the path that skips the loop cannot be executed, and the 8th independent path is obtained instead from a loop traversed several times (P4).
- No other path is infeasible in the original code: zone, express and parcels are independent of one another.

How the loop is tested

Table 21: Loop tests for N9–N10 (the number of iterations equals the number of started kilograms)

Iterations	Test (weight, zone, express, parcels)	Expected	Comment
0	Not possible	—	Infeasible, as explained above
1	0.1, A, No, 5 and 1.0, A, No, 5	₹40	Smallest loop count; 1.0 checks that w = 0 stops the loop
2	1.1, A, No, 5	₹80	Second iteration after a fractional remainder
Typical	25.3, C, No, 10 (26 iterations)	₹2,106	Typical value
Maximum	50.0, A, No, 5 (50 iterations)	₹2,000	Largest loop count that the code allows
Maximum + 1	50.1, A, No, 5	INVALID	Line 1 stops the loop count from exceeding 50

Is V(G) = 8 enough?

No. Eight tests exercise each decision outcome once, but they do not exercise every combination. Counting the logical paths (ignoring the number of loop iterations): 3 zones × (no express with or without discount: 2, express up to 30 kg with or without discount: 2, express above 30 kg: 1) = 15 paths, plus the INVALID path, giving 16. The eight basis tests cover 7 of the 16 (44%). I accept this because, after line 8, the zone is never read again, so paths that differ only in the zone behave the same way and a test for each zone with one other path is enough. Basis-path testing also cannot find the invalid-zone defect, because the missing branch is not a path in the graph. Hence V(G) is the lower limit on tests for structural coverage, not a sufficient number.

TASK 9 – Judge Test Adequacy and Coverage 3 Marks
CO2 | Bloom's Level: K5 – Evaluate

Management wants a single coverage number to decide whether testing is "enough".

Which coverage criterion is adequate for a module that calculates money for 40,000 parcels a day?

Discuss:

- **Statement, branch, condition and multiple-condition coverage for shipCharge**
- **The coverage achieved by your path tests from Task 8**
- **The additional tests needed to reach the next stronger criterion**
- **Whether 100 % coverage would have found the invalid-zone defect**

Your recommended criterion must be defended against cost and risk.

Evidence Expected
A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.

Ans:

Which Coverage Criterion Is Adequate for shipCharge?

Coverage measures how much of the code a test set has executed. I define the four criteria for shipCharge, measure the eight basis-path tests of Task 8 (P0–P7) against them, and decide which criterion is adequate for a module that prices 40,000 parcels a day.

Table 22: Coverage criteria for shipCharge and the coverage reached by the basis-path tests (P0–P7)

Criterion	What it demands in shipCharge	Smallest test set	Coverage achieved by P0–P7	Extra tests needed
Statement	All 19 lines executed	3 tests, for example (0,A,No,5), (35,B,Yes,5), (10,C,Yes,10)	19 / 19 = 100%	None
Branch (decision)	Both outcomes of 7 decisions = 14 outcomes	4 tests (the three above plus (10,A,No,5))	14 / 14 = 100%	None
Condition	Both outcomes of every condition: 8 conditions (line 1 has two) = 16 outcomes	5 tests (the four above plus weight 51)	15 / 16 = 93.8% ("weight > 50" is never true; P1 uses weight 0)	1 test: weight 51 (or 50.1)
Multiple-condition	All feasible combinations at line 1 (3 of 4; "≤ 0 and > 50" is impossible) plus the 14 outcomes elsewhere = 17	5 tests	16 / 17 = 94.1% (the combination false, true is missing)	The same one test
Path (logical paths)	All 16 logical paths (Task 8)	16 tests	7 / 16 = 44%	9 more

- **Statement coverage is too weak.** Three tests reach 100% and yet never price a zone A parcel, never take the non-express branch and never take the "no discount" branch. A fault in any of those would not be noticed.
- **The additional tests for the next stronger criterion:** adding the single test "weight 51" lifts the eight basis-path tests to 100% condition coverage and to 100% multiple-condition coverage at the same time, because line 1 is the only compound condition in the module. Test B6 (weight 50.1) of Task 4 already fulfils this requirement.

Would 100% coverage have found the invalid-zone defect?

No. With the 8 tests of Table 20 plus weight 51, statement, branch, condition and multiple-condition coverage are all 100%, and none of those tests uses a zone other than A, B or C. The defect is a missing branch: the code has no statement for "invalid zone", so there is nothing for a coverage tool to report as unexecuted. The corrected pseudocode (with an added "else return INVALID") would be covered by exactly the same tests. Coverage measures how much of the code that exists has been exercised; it cannot measure whether the code that should exist is present.

Recommended criterion

- **Branch coverage and condition coverage, both at 100%** (nine tests in all, with multiple-condition coverage included at no extra cost because the module has a single compound condition). It costs one test more than basis-path testing, whereas path coverage needs 9 more and adds almost nothing because the zone never influences later decisions.
- **It must be paired with 100% of the equivalence classes and boundaries of Tasks 3–4**, which is the black-box measure that catches omissions such as D1. Management asked

for one number, but a single coverage figure is misleading for a module that handles money: the defensible report is "branch 100%, condition 100%, specification classes 12/12, boundaries 11/11".

- **Risk and cost:** at 40,000 parcels a day, one untested branch can misprice thousands of parcels before it is noticed; the criteria above cost about 25 tests that run in seconds.

TASK 10 – Compare Black-Box and White-Box Effectiveness 4 Marks
CO2 | Bloom's Level: K5 – Evaluate

You now have black-box tests (Tasks 3–6) and white-box tests (Tasks 7–9).

Which approach actually found more of the important defects in shipCharge?

Discuss:

- **The defects found by black-box tests only, white-box tests only, and both**
- **Reasons why each approach missed what it missed**
- **Overlapping tests that can be removed without losing defect-finding power**
- **Whether this result would hold for other modules**

You are expected to interpret the comparison, not just count tests.

Evidence Expected

A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.

Ans:

Black-Box or White-Box: Which Found More?

To compare the approaches by evidence rather than by opinion, I used the one real defect (D1) and nine faults that I seeded into the corrected code, each a typical mistake in a money module (a wrong comparison operator, a wrong rate or factor, a wrong loop condition, a misplaced discount). I ran four test sets against all ten versions: the ECP tests (7), the BVA tests (11), the decision-table tests (8) and the basis-path tests with the weight 51 test (9). A set "finds" a fault if at least one of its tests returns a result different from the specification.

Table 23: Faults found by each test set (✓ = found, – = not found)

Fault	Description	ECP (7)	BVA (11)	Decision table (8)	Basis paths (9)	Found by
D1	Unknown zone charged ₹40/kg (real defect)	✓	–	–	–	Black-box only
M1	Line 1: weight < 0 instead of weight <= 0	–	✓	–	✓	Both
M2	Line 1: weight >= 50 instead of weight > 50	–	✓	–	–	Black-box only
M3	Line 14: weight >= 30 instead of weight > 30	–	✓	–	–	Black-box only
M4	Line 17: parcels > 10 instead of parcels >= 10	✓	✓	✓	✓	Both

Fault	Description	ECP (7)	BVA (11)	Decision table (8)	Basis paths (9)	Found by
M5	Line 16: express factor 1.25 instead of 1.5	✓	✓	✓	✓	Both
M6	Line 5: zone B rate 50 instead of 60	✓	–	✓	✓	Both
M7	Line 7: zone C rate 80 instead of 90	✓	–	✓	✓	Both
M8	Line 10: while w >= 0 instead of w > 0	✓	✓	✓	✓	Both
M9	Discount skipped when weight > 30	–	–	✓	–	Black-box only
Total found		**6 / 10**	**6 / 10**	**6 / 10**	**6 / 10**	

Interpretation

- **Black-box tests only: D1, M2, M3, M9 (4 faults).** The basis-path tests missed D1 because the branch does not exist in the code; they missed M2 and M3 because they use weights such as 35 and 51, far from the edges at 50 and 30, which satisfies every decision without testing the exact comparison; and they missed M9 because they never combine a heavy parcel with a discount.
- **White-box tests only: none.** Every fault that the basis-path tests found was also found by at least one black-box set (M1, M4–M8). White-box analysis did contribute differently: reading the CFG showed that the zero-iteration path is infeasible and that N4/N6 have no INVALID edge, but this came from inspection, not from coverage.
- **Both: M1, M4, M5, M6, M7, M8 (6 faults).**
- **Why each approach missed what it missed:** black-box sets derived from one technique miss what that technique does not vary (ECP does not test edges, BVA uses only zone A, the decision table uses weights 10 and 35); white-box tests satisfy structure with the cheapest data, so the data is rarely on an edge. No single set exceeds 6 of 10, while the union reaches 10 of 10.

Overlapping tests that can be removed

Table 24: Removing overlap

Test sets	Tests
Designed in Tasks 3–9 (ECP 7 + BVA 11 + decision table 8 + basis paths 9)	35 tests
After removing tests with identical inputs (for example (0,A,No,5), (1.0,A,No,5), (10,A,No,10), (35,A,Yes,5) were each designed twice)	31 tests
After removing tests that find nothing the others do not and that test no distinct rule (T1 duplicates B10; T4 duplicates B1; T5 duplicates B6; T7 duplicates B8; all 9 basis-path tests duplicate ECP, BVA or decision-table tests; 6 of the 8 decision-table tests duplicate others)	**16 tests**
Faults found by the 16 tests	**10 / 10, the same as the union of all 35**

In the 16 remaining tests six tests are the only ones that find a particular fault (T3 for M7, T6 for D1, B1 for M1, B5 for M2, B7 for M3 and the R4 test for M9). The other ten are kept because each checks a distinct specification rule or boundary side (for example B6 for 50.1 and B8 for 30.1), not because they add to the fault count in this experiment.

Would the result hold for other modules?

Not necessarily. Here the module is small (V(G) = 8), its logic is simple and its main defect is an omission, which favours black-box testing. White-box tests would be expected to find more in modules with many branches, complex loops, or code written without a clear specification (for example, error handling or defensive code), where the code contains behaviour the specification never mentions. The tracking module has the opposite shape: it is a state model, so black-box transition tests are the natural choice. The faults are also ones I seeded, so the numbers show the pattern, not a measure of real defect rates.

TASK 11 – Recommend and Defend the Final Test Suite 4 Marks
CO2 | Bloom's Level: K5 – Evaluate

Based on your analysis, recommend the final regression test suite for SwiftShip.

Discuss:

- **The test cases you include and the technique each comes from**
- **Why this suite is better than the alternatives**
- **The coverage and the rule combinations it achieves**
- **The number of test cases and the effort needed to run them**
- **Known gaps and risks that remain**

Your recommendation must be based on evidence from your own test design. The largest suite is not automatically the best.

Evidence Expected

A technically justified final test suite supported by black-box, white-box and coverage evidence.

Ans:

The Final Regression Suite for SwiftShip

The final suite keeps one test for each distinct rule, boundary side and rule combination, and drops every test that duplicates another. It consists of 16 shipCharge tests (Table 25) and the 13 tracking tests of Table 15, 29 tests in all.

Table 25: Final shipCharge tests (weight, zone, express, parcels) and the technique each comes from

ID	Input	Expected	Technique	Purpose
F1	12.5, B, Yes, 12	₹1,053	ECP (T2), decision rule R6	Zone B, express and discount together
F2	25.3, C, No, 10	₹2,106	ECP (T3), rule R2	Zone C rate; started-kilogram rounding
F3	10.0, D, No, 3	INVALID	ECP (T6)	Invalid zone (defect D1)

ID	Input	Expected	Technique	Purpose
F4	0, A, No, 5	INVALID	BVA (B1)	Lower weight limit
F5	0.1, A, No, 5	₹40	BVA (B2)	Smallest valid weight
F6	1.0, A, No, 5	₹40	BVA (B3)	Exactly one kilogram
F7	1.1, A, No, 5	₹80	BVA (B4)	Started-kilogram edge
F8	50.0, A, No, 5	₹2,000	BVA (B5)	Upper weight limit
F9	50.1, A, No, 5	INVALID	BVA (B6)	Above the upper limit
F10	30.0, A, Yes, 5	₹1,800	BVA (B7), rule R5	Express limit (allowed)
F11	30.1, A, Yes, 5	NOT_ALLOWED	BVA (B8), rule R7	Express limit (rejected)
F12	30.1, A, No, 5	₹1,240	BVA (B9), rule R3	Express rule must not affect other parcels
F13	10.0, A, No, 9	₹400	BVA (B10), rule R1	Just below the discount threshold
F14	10.0, A, No, 10	₹360	BVA (B11), rule R2	At the discount threshold
F15	35, B, No, 12	₹1,890	Decision table, rule R4	Heavy parcel with discount
F16	35, A, Yes, 12	NOT_ALLOWED	Decision table, rule R8	Discount must not change a rejection

The tracking tests are F17 to F29 (ST1–ST7 and X1–X6 of Table 15), all derived by state-transition testing.

Why this suite is better than the alternatives

Table 26: The final suite compared with the alternatives

Alternative	Tests	Faults found (of 10)	Weakness
1,000 random tests	1,000	Unknown; the invalid zone is never generated and edges are hit by chance (Table 3)	Needs a reference model; no guarantee about edges
ECP only	7	6	Misses wrong operators at 0, 30 and 50 (M1–M3) and the heavy-parcel discount (M9)
BVA only	11	6	Uses only zone A, so zone-rate faults and the invalid zone are missed
Basis paths only	9	6	Misses D1, M2, M3 and M9
Everything designed (Tasks 3–9)	35 (31 distinct)	10	Duplicates; costs more to maintain
Final suite	**16 (+13 tracking)**	**10**	Known gaps below

Coverage and rule combinations achieved

- **Black-box:** 12 of 12 equivalence classes (Table 5); 11 of 11 selected boundary values; 8 of 8 decision-table rules (R1: F13; R2: F14, F2; R3: F12; R4: F15; R5: F10; R6: F1; R7: F11; R8: F16); 8 of 8 valid and 6 selected invalid tracking transitions.

- **White-box:** 19 of 19 statements; 14 of 14 branch outcomes; 16 of 16 condition outcomes; all 17 multiple-condition combinations (F4 for "≤ 0", F9 for "> 50"). Logical paths covered: 8 of 16.
- **Faults:** 10 of 10 seeded and real faults found (Table 23).

Number of tests and effort

The 29 tests are automated as a table of inputs and expected results for shipCharge and as scripted event sequences for tracking. I estimate about 2 seconds per test including set-up, so a full run takes roughly 1 to 2 minutes. Run by hand, at an assumed 3 minutes per test, the same suite would take about 87 minutes. Maintaining the suite needs one table row per rule change.

Known gaps and risks

- Weights with more than one decimal, negative parcel counts, lower-case zone names and the rounding of money are not defined by the specification, so no test has an agreed expected result. They must be clarified before they can be tested.
- Eight of the 16 logical paths are not executed (zone variants of covered paths). This relies on the argument that zone is not read again after line 8, which stops being true after the change in Task 12.
- Of the 56 illegal tracking transitions, 6 are tested; the other 50 rely on the claim that they have only cosmetic consequences.
- The suite checks logic, not performance, concurrency or integration with the booking system at 40,000 parcels a day; these need separate tests.
- Time figures are estimates; actual run times should be recorded in the first automated run.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Test Suite 8 Marks
CO2 | Bloom's Level: K5 – Evaluate

Changed Requirement

SwiftShip will add a fuel surcharge of 8 % on Zone C parcels and a weekend delivery option that adds ₹50 per parcel.

The developers have changed shipCharge by adding two new decisions after line 16.

The suite must still run within 30 minutes before every release.

Re-evaluate your original solution in response to these changed conditions.

You may: RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis – 2 Marks
CO2 | Bloom's Level: K5 – Evaluate

Discuss:

- **What has changed in the specification and the code**

- **Which equivalence classes, boundaries and decision-table rules are affected**
- **How the control flow graph and cyclomatic complexity change**
- **Why the 30-minute limit matters**

Evidence Expected

A clear analysis of how the changes affect the original test design.

Ans:

Impact Analysis of the Fuel Surcharge and Weekend Delivery

The specification and the code both change, and the new rules interact with rules that were previously independent. Because the specification does not say in which order the new charges are applied, I state my assumption: the developers inserted the two decisions after line 16 and before the discount, so (S5) zone C charges are multiplied by 1.08 after the express factor, (S6) a weekend delivery adds ₹50, and (S4) the discount applies to the final charge including both. Results are rounded to paise. The order must be confirmed with the product owner, because it changes the price of every express, weekend or loyal-customer parcel.

Table 27: What has changed

Aspect	Original	Changed
Rules	S1–S4	S1–S4 plus S5 (8% fuel surcharge for zone C) and S6 (₹50 weekend delivery)
Inputs of shipCharge	weight, zone, express, parcels	The same plus a weekend flag
Code	19 lines; zone read once (lines 4–7)	Two new decisions after line 16: "zone == C" (surcharge) and "weekend" (₹50); zone is now read twice
Expected results	Zone C charges are base rate × kilograms	Every zone C charge is 8% higher; any parcel with weekend delivery is ₹50 higher before discount
Time limit	Not binding	The full suite must finish within 30 minutes before every release

Which classes, boundaries and rules are affected

- **Equivalence classes:** two new classes (weekend = No, weekend = Yes), so 12 become 14. Class Z3 (zone C) now has a different expected result in every test that uses it.
- **Boundaries:** no new numeric edge appears, and the edges 0, 0.1, 30, 30.1, 50 and 50.1 remain. For zone C, the expected results at those edges change, and the discount edge (parcels 9 and 10) now also has to be checked together with the ₹50 charge, to confirm that the discount applies after it.
- **Decision table:** conditions grow from 3 (express, weight > 30, parcels ≥ 10) to 5 (plus zone C and weekend). There are $2^5 = 32$ combinations; the 8 in which express and weight > 30 are both true all give NOT_ALLOWED and collapse into one rule, leaving 24 + 1 = 25 distinct rules instead of 7.

Table 28: Control flow graph and complexity before and after the change

Measure	Original	Changed
Statements (lines)	19	23
Decision nodes	7	9
Nodes / edges of the CFG	18 / 24	22 / 30
V(G) = E − N + 2	24 − 18 + 2 = 8	30 − 22 + 2 = 10 (predicates 9 + 1)
V(G) at condition level	9	11
Branch outcomes	14	18
Logical paths	16	28 (3 zones × 9 + INVALID)
Feasible independent paths	8	9 of the 10 structural paths

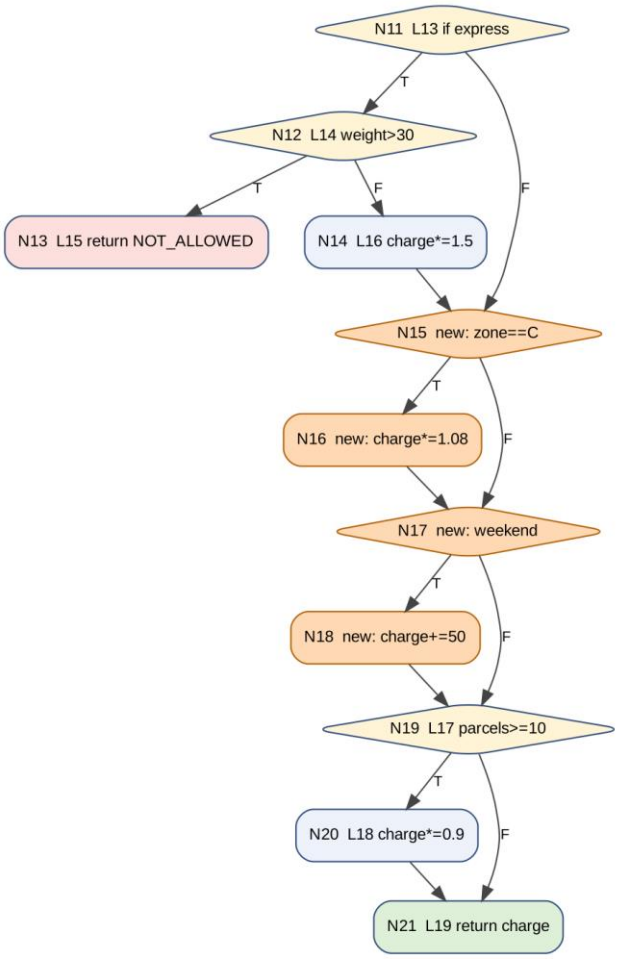


Figure 3: Changed part of the control flow graph (new nodes in orange; N1–N10 are unchanged)

The new decision “zone == C” tests the same variable as line 6, so the two predicates are perfectly correlated: the combination “line 6 true and surcharge false” (or the reverse) can never occur. One of the 10 structural basis paths is therefore infeasible, and at most 9 independent paths can be tested. Statement and branch coverage can still reach 100%, but 100% path coverage cannot.

Why frequent releases and the 30-minute limit matter

A release every two weeks means the full regression suite runs at least 26 times a year, and every run must fit within 30 minutes before the release can go out. A hand-run suite of 38 tests at about 3 minutes each (my assumption) would take 114 minutes, nearly four times the limit, so the suite has to be automated and every added test must pay for its running time. The limit also forces selection: the new rules cannot be tested by crossing every factor (3 zones × 2 express × 2 weekend × 2 discount × boundary values), so the new tests have to be derived by a technique that keeps the number small (Task 12C).

B. Evaluation of the Original Suite – 2 Marks
CO2 | Bloom's Level: K5 – Evaluate

Evaluate whether your final test suite remains adequate.

Discuss:

- **Which existing tests are still valid, need new expected results, or are obsolete**
- **Which new paths and rule combinations are not covered**
- **Whether the coverage criterion can still be met**

Evidence Expected

A technically supported evaluation of the original suite under the new conditions.

Ans:

Is the Original Suite Still Adequate?

I ran the original 16 shipCharge tests (with the extra weekend = No argument) against the new specification and assessed which still hold, which need a new expected result and which combinations have never been run.

Table 29: Status of the existing tests under the changed rules

Tests	Status	Reason
F1, F3–F16 (15 tests)	Still valid; only the new weekend = No argument is added to the call	They use zone A or B, are INVALID or NOT_ALLOWED, and weekend = No adds nothing
F2 (25.3, C, No, 10)	Valid but needs a new expected result: ₹2,106 becomes ₹2,274.48	26 × 90 = 2,340; × 1.08 = 2,527.20; × 0.9 = ₹2,274.48
Obsolete tests	None	No rule was removed
Tracking tests F17–F29 (13 tests)	Unchanged	The tracking module is not affected by the change; I assume that weekend delivery adds no new tracking state, which should be confirmed

Zone C also exposes a testing risk: F2 now carries paise, so the expected result needs a rounding rule that the specification does not give.

What is not covered

Table 30: New paths and rule combinations that the original suite does not reach

Gap	Why it matters
Weekend = Yes is never executed (22 of 23 statements, 17 of 18 branch outcomes covered)	The ₹50 charge could be missing, wrong or applied twice without any test failing
Zone C together with express	Whether the 8% applies to the express price (as assumed) or to the base price changes the charge
Weekend together with the discount	The ₹50 inside or outside the 10% discount changes the amount by ₹5 on every discounted weekend parcel
Weekend together with NOT_ALLOWED	A rejected express booking must not be charged, even on a weekend
Zone C parcels above 30 kg without express	The surcharge must not depend on the weight limit or on express
Misplaced insertion	Line 16 sits inside the "if express" block; if the new decisions were inserted at that indentation, the surcharge and the fee would apply only to express parcels, and a plain zone C or weekend parcel would be undercharged
Correlated predicates	Two decisions on zone C cannot be tested independently; 100% path coverage is impossible

Can the coverage criterion still be met?

Yes. The criterion I recommended (100% branch and condition coverage plus all equivalence classes, boundaries and decision-table rules) remains achievable: the weekend = Yes branch needs one test to reach 100% statement and branch coverage. The number of tests needed is determined by the combinations in Table 30, not by coverage, because coverage can reach 100% while the order of operations is still wrong. That is why the new tests below are derived from the decision table and not only from coverage.

C. Adaptation or Redesign – 2 Marks
CO2 | Bloom's Level: K5 – Evaluate

Decide whether to:

Retain, modify or redesign the test suite.

If you modify or redesign it:

Discuss:

- **Describe the new and changed test cases**
- **Explain which techniques you used to derive them**
- **Show the effect on coverage and the number of test cases**

If you retain the original suite, provide sufficient justification for why no change is necessary.

Evidence Expected
An appropriate test-design response to the changed requirement with clear reasoning.

Ans:

Decision: Modify the Test Suite

I modify the suite. I do not retain it unchanged, because one expected result is wrong, the weekend branch is never run and 7 of the 24 pairs of rule values are never combined (Table 32). I do not redesign it, because the structure of the suite (ECP, BVA, decision table and state transitions) is still sound: 15 of the 16 shipCharge tests and all 13 tracking tests remain valid without change. What I add is a small set of tests for the interaction of the new rules.

How the new tests were derived

- **Decision table, reduced by pairwise selection.** The four rules that can interact are zone C, express, weekend and discount. Testing all 16 combinations of these four (and more with weights) is unnecessary; I choose the tests so that every pair of values of any two factors occurs at least once (6 factor pairs × 4 value pairs = 24 pairs), which also captures the order of operations.
- **Boundary value analysis** for the express limit with the largest charge (30.0 kg, zone C, express, weekend).
- **Equivalence classes:** the heavy zone C non-express parcel (weight above 30 without express), and the NOT_ALLOWED class combined with weekend.
- **White-box:** the new decisions have outcome pairs that must each be executed: N3 and N4 take the weekend branch; N1 takes the surcharge branch with no weekend; the original tests take both "false" outcomes.

Table 31: New and changed tests (weight, zone, express, weekend, parcels)

ID	Input	Technique	Expected result and calculation	What it checks
F2 (changed)	25.3, C, No, No, 10	Existing test	2,340 × 1.08 = 2,527.20; × 0.9 = ₹2,274.48	Surcharge with discount
N1	1, C, No, No, 5	Decision table, ECP	90 × 1.08 = ₹97.20	Surcharge applies without express (catches misplaced insertion)
N2	10, C, Yes, No, 5	Decision table (pair C × express)	900 × 1.5 = 1,350; × 1.08 = ₹1,458.00	Surcharge on the express price
N3	10, A, No, Yes, 5	Decision table (weekend alone)	400 + 50 = ₹450.00	Weekend fee without express
N4	10, B, Yes, Yes, 5	Decision table (express × weekend)	600 × 1.5 = 900; + 50 = ₹950.00	Weekend fee with express, no surcharge
N5	10, C, Yes, Yes, 10	Decision table (all four rules)	1,350 × 1.08 = 1,458; + 50 = 1,508; × 0.9 = ₹1,357.20	Order: discount applies last
N6	10, C, No, Yes, 12	Decision table (C × weekend × discount)	900 × 1.08 = 972; + 50 = 1,022; × 0.9 = ₹919.80	Weekend fee inside the discount
N7	35, C, Yes, Yes, 5	ECP (E3) × weekend	NOT_ALLOWED	Rejection is not overridden by weekend
N8	30.0, C, Yes, Yes, 5	BVA at the express limit	2,700 × 1.5 = 4,050; × 1.08 = 4,374; + 50 = ₹4,424.00	Largest allowed express charge
N9	35, C, No, No, 5	ECP (heavy, non-express)	3,150 × 1.08 = ₹3,402.00	Surcharge does not depend on weight

Together with the existing tests, these cover all 24 pairs of the four factors (for example: zone C × weekend in N6, express × weekend in N4, weekend × discount in N5, zone C × express in N2). A change of order or of rate would be caught as follows: I seeded six faults into the new code (surcharge only inside the express block; weekend fee added after the discount; 0.8% instead of 8%; weekend fee skipped for express parcels; ₹5 instead of ₹50; surcharge also applied to zone B). The updated original suite alone found 3 of the 6 (the surcharge faults, through its zone B and zone C tests); the original suite plus N1–N9 found all 6.

Table 32: Effect of the change on coverage and on the number of tests

Measure	Original suite on the original code	Original suite on the changed code	Modified suite on the changed code
shipCharge tests	16	16	25
Tracking tests	13	13	13
Total tests	29	29	38
Statement coverage	19 / 19 = 100%	22 / 23 = 95.7%	23 / 23 = 100%
Branch outcomes	14 / 14 = 100%	17 / 18 = 94.4%	18 / 18 = 100%
Condition outcomes	16 / 16 = 100%	19 / 20 = 95.0%	20 / 20 = 100%
Pairs of rule values combined	—	17 / 24 = 70.8%	24 / 24 = 100%
Seeded faults in the new code found	—	3 / 6	6 / 6
Automated run time (estimate)	1–2 min	1–2 min	About 2 min

Revised role of the test suite

The suite keeps the same structure: boundary and rule tests run on every build, the decision-table and pairwise tests run before each release, and the tracking tests run unchanged. Each new fee or surcharge from now on is added by one new decision-table condition and the pairs it creates, instead of redesigning the suite.

D. Evidence-Based Defence – 2 Marks
CO2 | Bloom's Level: K5 – Evaluate

Defend your final test suite using evidence.

Use:

Discuss:

- **Equivalence classes and boundaries**
- **Decision-table combinations**
- **Cyclomatic complexity and basis paths**
- **Coverage achieved**
- **Execution time limit**

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final test suite integrating test design evidence and the changed constraints.

Ans:

Defending the Final Test Suite

Table 33: Evidence in support of the decision to modify the suite

Evidence	What it shows	Effect on my decision
Equivalence classes and boundaries	Classes grow from 12 to 14; the six weight boundaries are unchanged; only the expected results for zone C change. 15 of 16 shipCharge tests remain valid	Keep the boundary and class tests; update one expected value
Decision-table combinations	Rules grow from 7 to 25; the old suite combines only 17 of 24 value pairs (70.8%) and misses the order of weekend fee, surcharge and discount	Add 9 pairwise tests derived from the table (N1–N9)
Cyclomatic complexity and basis paths	V(G) rises from 8 to 10; only 9 independent paths are feasible because the zone is tested twice	Nine new tests exercise both new decisions in all outcomes; path coverage cannot reach 100%
Coverage achieved	The old suite falls to 95.7% statement and 94.4% branch coverage; the modified suite is back to 100% / 100% / 100% (statement, branch, condition)	The coverage criterion of Task 9 is kept
Execution time limit	38 tests at about 2 seconds each run in about 2 minutes. By hand (about 3 minutes each) they would take 114 minutes, nearly four times the limit	Automation is mandatory; there is room for hundreds of additional tests before the 30 minutes are used up
Seeded faults	Old suite 3 of 6; modified suite 6 of 6	Redesign is unnecessary; adding tests is sufficient

Reasoning. If the suite were retained, the weekend fee would never be tested and one expected result would be wrong, so a fault in the new code could reach customers on any of the 40,000 parcels booked each day. If I redesigned the suite, I would discard 15 of 16 working tests and the structure that found all ten faults in Task 10, for no gain. Modification adds only nine tests and about one extra minute of run time.

Risks I accept: (1) the order of the new charges (surcharge after express, weekend fee before the discount) is my assumption, and the tests encode it, so if the product owner decides otherwise, N2, N5 and N6 change; (2) the specification does not define rounding of paise, so the expected results use two decimals; (3) the 50 untested illegal tracking transitions and the zone variants of the logical paths remain untested; (4) the time figures are estimates until the suite has actually been run.

Review trigger: I would reconsider the decision, and redesign towards table-driven combinatorial tests, if V(G) rises above about 15, if more than two further charge rules are added (the pairs would exceed what hand-picked tests can cover), or if the measured run time exceeds about 20 of the 30 minutes.

MARK DISTRIBUTION

Task	Marks
Task 1 – Select Test Case Design Strategies	5

Task	Marks
Task 2 – Evaluate Random Testing	2
Task 3 – Design and Evaluate Equivalence Classes	6
Task 4 – Design and Evaluate Boundary Value Tests	4
Task 5 – Apply Other Black-Box Techniques	3
Task 6 – Design and Evaluate State-Transition Tests	3
Task 7 – Construct the Control Flow Graph	4
Task 8 – Evaluate Path Testing and Cyclomatic Complexity	4
Task 9 – Judge Test Adequacy and Coverage	3
Task 10 – Compare Black-Box and White-Box Effectiveness	4
Task 11 – Recommend and Defend the Final Test Suite	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	50